UNIVERSIDAD DE SAN BUENAVENTURA
Tecnología en Desarrollo de Software · VI semestre
Programación Avanzada y Aplicada

Consumo de la API REST

Solicitud, respuesta, códigos HTTP, reintentos y seguridad

Proyecto final: **AsistenteIA**, asistente académico móvil con .NET MAUI y una API REST de IA

Autores: Jeferson Wilderman González Tenjo, María Alejandra Gómez y José Leandro Ocampo
Docente: Mg. Sandra Esperanza Sánchez Barragán
Fecha: 16 de octubre de 2026
Versión evaluada: rama `feat/rediseno-interfaz` (commit 6a15fc1)

Introducción

AsistenteIA envía cada consulta a una API REST de inteligencia artificial compatible con Chat Completions de OpenAI, mediante una solicitud POST con cuerpo JSON y autenticación Bearer. Este documento detalla la solicitud, la respuesta, todos los códigos HTTP que maneja la app y cómo se protegen las credenciales.

La app admite dos proveedores con el mismo formato (Chat Completions de OpenAI). Se eligen en **Configuración → Proveedor de IA** (`Models/AiProvider.cs`):

Proveedor	URL base	Modelo por defecto	Claves
Pollinations	`https://gen.pollinations.ai/v1/`	`openai/gpt-5.4-nano`	https://enter.pollinations.ai/keys (sk_...)
Google Gemini	`https://generativelanguage.googleapis.com/v1beta/openai/`	`gemini-3.8-flash`	https://aistudio.google.com/apikey (AIza...)

Cada proveedor guarda su propia clave en SecureStorage. Si la clave pegada tiene el prefijo de otro proveedor, la app selecciona ese proveedor automáticamente.

Solicitud

```
POST https://gen.pollinations.ai/v1/chat/completions
Authorization: Bearer <clave del usuario, leída de SecureStorage>
Content-Type: application/json; charset=utf-8

{
  "model": "openai/gpt-5.4-nano",
  "messages": [
    { "role": "system", "content": "Eres un asistente académico. Responde en español de forma clara y ordenada, indica cuando no estés seguro y no inventes fuentes ni datos." },
    { "role": "user", "content": "Explica el siguiente concepto para un estudiante universitario, con una definición, un ejemplo y un error común: API REST" }
  ]
}
```

En C#, el cuerpo es un `ChatRequest` serializado con `JsonContent.Create`. Los nombres JSON se fijan con `[JsonPropertyName]` (`Models/ChatModels.cs`).

Respuesta exitosa (200)

La app solo mapea la parte que necesita:

```
{
  "choices": [
    { "message": { "role": "assistant", "content": "Una API REST es..." } }
  ]
}
```

`ReadFromJsonAsync<ChatResponse>()` → `Choices[0].Message.Content`.

Respuestas de error

Formato real devuelto por el proveedor (comprobado con una clave inválida):

```
{"success":false,"error":{"message":"A valid API key is required. Get one at https://enter.pollinations.ai/keys",
"code":"UNAUTHORIZED","timestamp":"2026-10-15T23:32:18.366Z"},"status":401}
```

Código / condición	Significado	Mensaje en la app	Prueba
200	Éxito	"Respuesta recibida. Verifique la información..."	P01

Código / condición	Significado	Mensaje en la app	Prueba
400 + "API key" (Google)	Clave inválida (Google responde 400 `INVALID_ARGUMENT` con un **arreglo** JSON)	"Error 400: la clave no es válida para este proveedor…"	—
404	Modelo inexistente para el proveedor	"Error 404: el modelo no existe para este proveedor…"	—
401 / 403	Clave ausente, inválida o sin permisos	"Error 401: la clave no es válida o no tiene permisos. Actualícela en Configuración."	P04
402	Saldo o presupuesto de la clave agotado	"Error 402: el saldo o presupuesto de la clave se agotó."	—
429	Demasiadas solicitudes o cuota agotada	"Error 429: demasiadas solicitudes o cuota agotada… (Proveedor: You exceeded your current quota…)"	A08
5xx	Falla o saturación del proveedor	"Error 503: el servicio de IA no está disponible… (Proveedor: This model is currently experiencing high demand…)"	A07
Otro 4xx	Solicitud rechazada	"Error N: " + mensaje del proveedor	—
Sin red (`IConnectivity`)	No hay internet	"Sin conexión a internet. Revise su red e intente de nuevo."	P03
`HttpRequestException`	DNS, TLS o conexión caída	"No fue posible contactar el servicio. Verifique su conexión."	P03
Tiempo agotado (CTS propio)	Se superó el tiempo configurado	"Tiempo de espera agotado (N s). Puede reintentar."	P05
Cancelación del usuario	Botón Cancelar	"Consulta cancelada por el usuario. Puede enviarla de nuevo."	P06
JSON inesperado	Formato distinto	"El servicio devolvió una respuesta con formato inesperado."	—

Formato de error de Google (comprobado con una clave falsa):

```
[{ "error": { "code": 400, "message": "Please pass a valid API key", "status": "INVALID_ARGUMENT" } }]
```

`AiService.ExtractErrorMessage` acepta ambos formatos (objeto o arreglo) y añade el mensaje del proveedor al error, por ejemplo: "(Proveedor: Please pass a valid API key)". Ese mensaje nunca incluye la clave.

Reintentos

Ante **503** o **429**, `SendChatAsync` reintenta hasta 2 veces, con esperas de 2 y 4 segundos, dentro del mismo tiempo máximo configurado. Si el usuario cancela, también se cancela la espera. Si los reintentos se agotan, se muestra el error con el motivo del proveedor.

Catálogo de modelos

`GET {URL base}models`. En Pollinations es público y la app filtra `category == "text"` y `paid_only == false`. En Gemini requiere la clave; la app quita el prefijo `models/` y descarta los modelos de voz, imagen, audio y embeddings. Si la consulta falla, se usa la lista local del proveedor (`AiService.GetModelsAsync`).

Seguridad del consumo

- La clave se envía solo en el encabezado `Authorization` sobre HTTPS y nunca en la URL.
- No se registra en consola ni en archivos, y no se muestra completa en pantalla (solo •••• y los últimos 4 caracteres).
- El repositorio no contiene claves (`git grep` de `sk_`/`pk_` en el código: sin resultados con claves reales).

Implementación (AiService.SendChatAsync)

```
    private async Task<ChatResponse?> SendChatAsync(ChatRequest body, CancellationToken userToken)
    {
        if (_connectivity.NetworkAccess != NetworkAccess.Internet)
            throw new AiServiceException(AiErrorKind.NoConnection,
                "Sin conexión a internet. Revise su red e intente de nuevo.");

        string? apiKey = await _keyService.GetAsync();
        if (string.IsNullOrWhiteSpace(apiKey))
            throw new AiServiceException(AiErrorKind.MissingKey,
                $"Configure la clave de {_settings.Provider.Name} en la pestaña Configuración.");

        // Tiempo de espera propio para distinguirlo de la cancelación del usuario.
        using var timeoutCts = new CancellationTokenSource(TimeSpan.FromSeconds(_settings.TimeoutSeconds));
        using var linkedCts = CancellationTokenSource.CreateLinkedTokenSource(userToken, timeoutCts.Token);

        var endpoint = new Uri(new Uri(_settings.Provider.BaseUrl), "chat/completions");

        try
        {
            // Los picos de demanda (503) y los límites momentáneos (429) suelen pasar en segundos:
            // se reintenta hasta 2 veces con espera creciente, dentro del mismo tiempo máximo.
            for (int attempt = 0; ; attempt++)
            {
                using var request = new HttpRequestMessage(HttpMethod.Post, endpoint);
                request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", apiKey);
                request.Content = JsonContent.Create(body);

                using HttpResponseMessage response = await _client.SendAsync(request, linkedCts.Token);
                if (response.IsSuccessStatusCode)
                    return await response.Content.ReadFromJsonAsync<ChatResponse>(cancellationToken: linkedCts.Token);

                bool transient = response.StatusCode is HttpStatusCode.ServiceUnavailable or HttpStatusCode.
TooManyRequests;
                if (!transient || attempt >= MaxRetries)
                    throw await CreateHttpErrorAsync(response, linkedCts.Token);

                await Task.Delay(TimeSpan.FromSeconds(2 << attempt), linkedCts.Token);
            }
        }
        catch (OperationCanceledException) when (timeoutCts.IsCancellationRequested && !userToken.
IsCancellationRequested)
        {
            throw new AiServiceException(AiErrorKind.Timeout,
                $"Tiempo de espera agotado ({_settings.TimeoutSeconds} s). Puede reintentar.");
        }
        catch (HttpRequestException ex)
        {
            throw new AiServiceException(AiErrorKind.NoConnection,
                "No fue posible contactar el servicio. Verifique su conexión.", ex.StatusCode, ex);
        }
        catch (JsonException ex)
        {
            throw new AiServiceException(AiErrorKind.BadResponse,
                "El servicio devolvió una respuesta con formato inesperado.", null, ex);
        }
    }
```

Evidencias

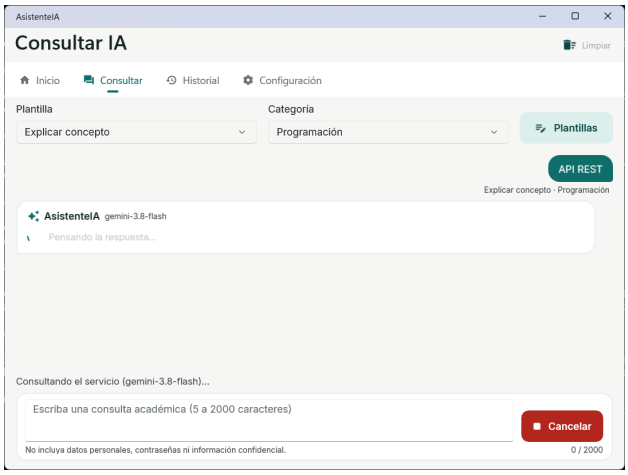


Figura 1. POST en curso (estado de carga).

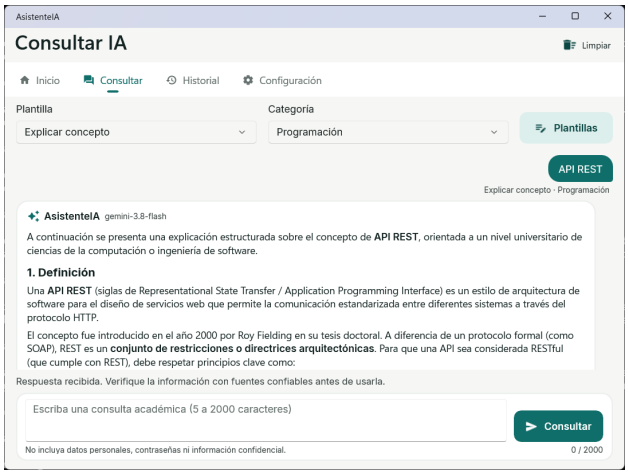


Figura 2. 200: respuesta recibida.

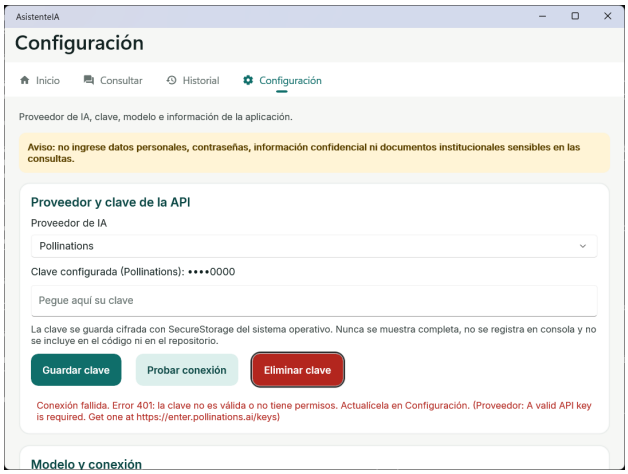


Figura 3. 401: clave inválida.

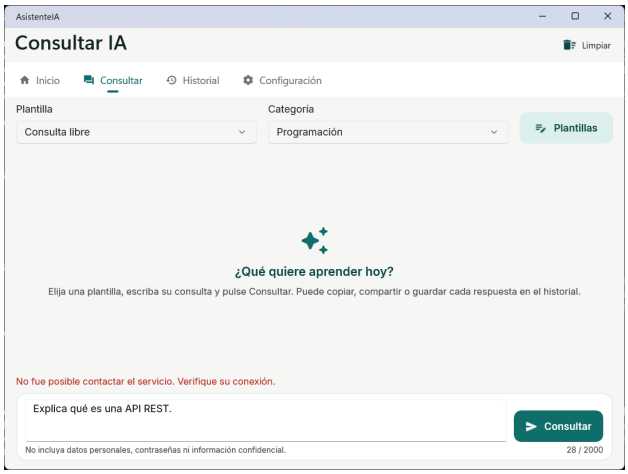


Figura 4. Sin conexión.

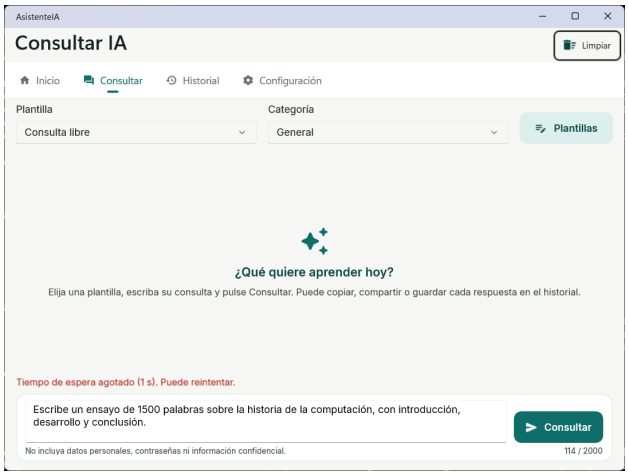


Figura 5. Tiempo agotado.

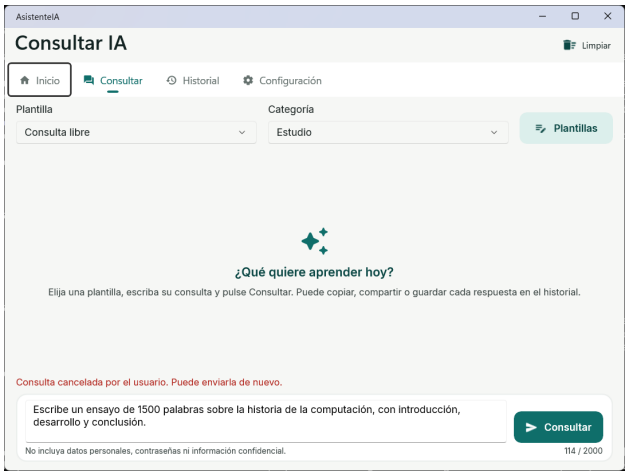


Figura 6. Cancelación del usuario.